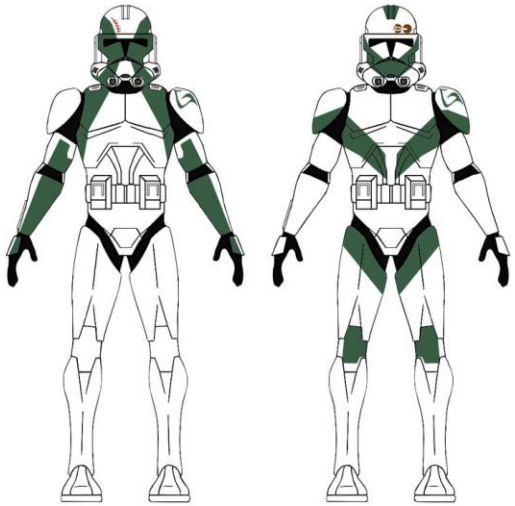
Порождающие паттерны. Прототип

Паттерн Прототип (Клон)
Prototype



Прототип — это порождающий паттерн проектирования, который позволяет копировать объекты, не вдаваясь в подробности их реализации.

Паттерн **Prototype** – позволяет задать различные виды (*классы-виды*) объектов-продуктов (*клонов*), через настройку состояния каждого нового созданного *клона*. Классификация *клонов-продуктов* производится на основании различия их состояний.

Проблема

1. Копирование путем создания пустого объекта того-же класса с последующим копированием его полей не всегда возможно, т.к. часть полей *private*.
2. Копирующий код станет зависим от классов копируемых объектов. Чтобы перебрать ***все*** поля объекта, нужно привязаться к его классу. Из-за этого невозможно копировать объекты, зная только их интерфейсы, а не конкретные классы.



Копирование «извне» не всегда возможно в реальности.

Решение

- Паттерн **Прототип** поручает создание копий самим копируемым объектам. Он вводит общий интерфейс для всех объектов, поддерживающих клонирование. Это позволяет копировать объекты, не привязываясь к их конкретным классам. Обычно такой интерфейс имеет всего один метод `clone`.
- Реализация этого метода в разных классах очень схожа. Метод создаёт новый объект текущего класса и копирует в него значения всех полей собственного объекта. Так получится скопировать даже *приватные* поля, так как большинство языков программирования разрешает доступ к приватным полям любого объекта текущего класса.
- Объект, который копируют, называется ***прототипом,*** полученный на его основе - ***клоном***. Когда объекты программы содержат десятки полей и сотни возможных конфигураций, прототипы могут служить своеобразной альтернативой созданию подклассов.

Предварительно заготовленные прототипы могут стать заменой подклассам.

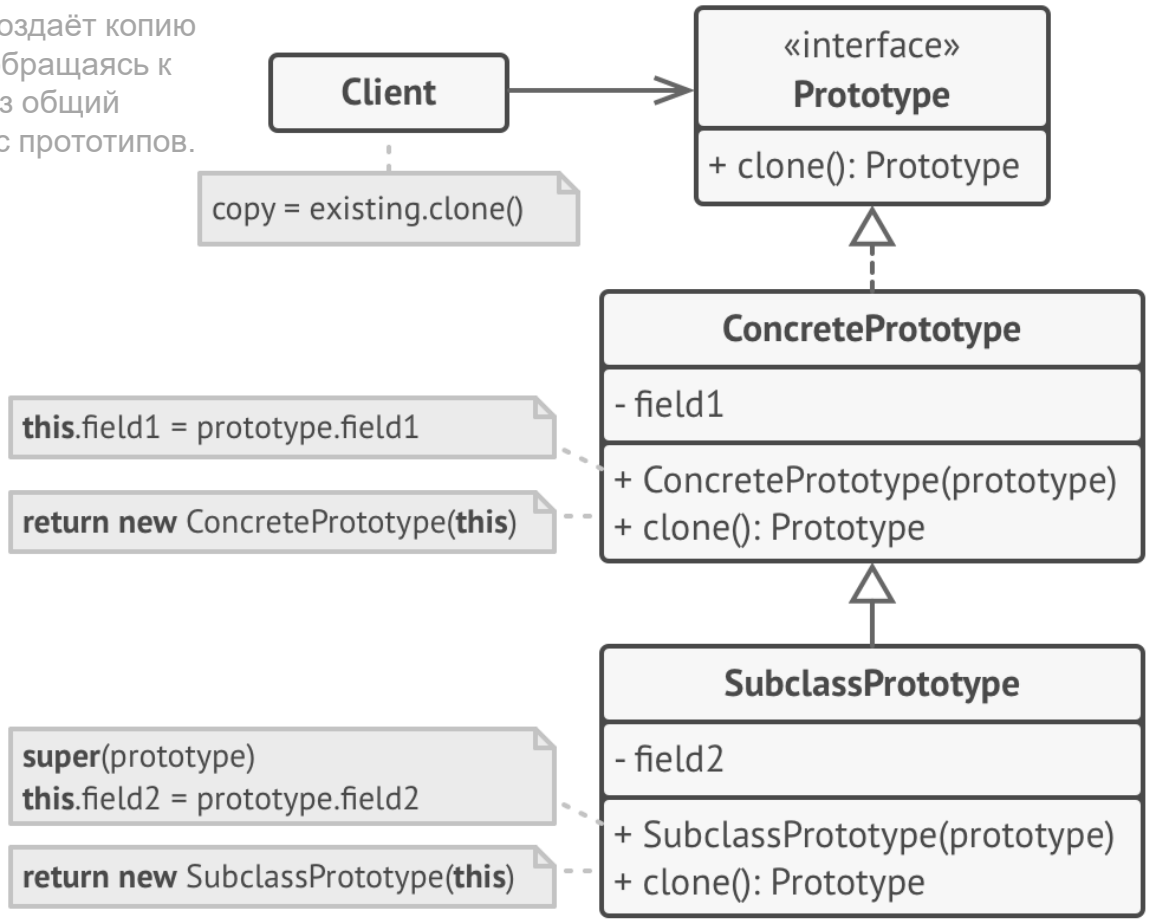
В этом случае все возможные прототипы заготавливаются и настраиваются на этапе инициализации программы.

Когда программе нужен новый объект, она создаёт копию из приготовленного прототипа.

Структура паттерна

Базовая реализация

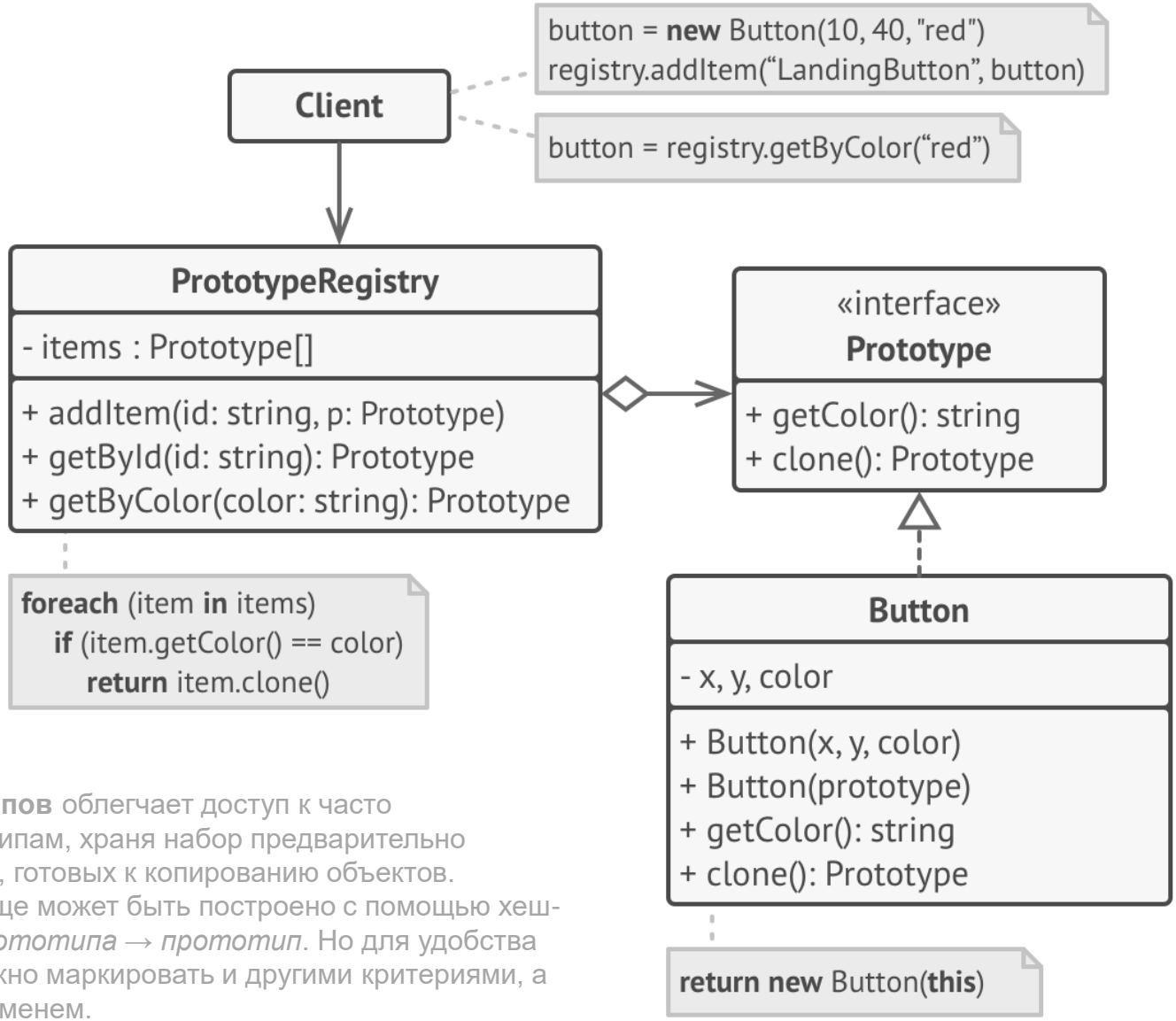
[3]**Клиент** создаёт копию объекта, обращаясь к нему через общий интерфейс прототипов.



[1]**Интерфейс прототипов** описывает операции клонирования. В большинстве случаев — это единственный метод clone.

[2]**Конкретный прототип** реализует операцию клонирования самого себя. Помимо копирования значений всех полей, здесь могут быть реализованы дополнительные операции, о которых не нужно знать клиенту. Например, клонирование связанных объектов, распутывание рекурсивных зависимостей и прочее.

Реализация с общим хранилищем прототипов

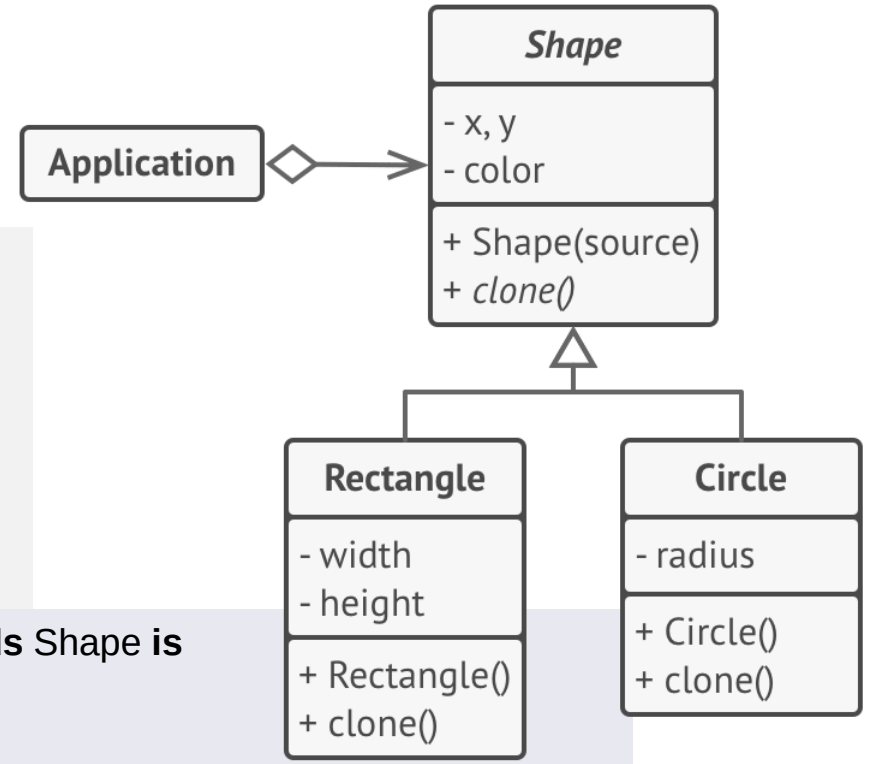


[1]**Хранилище прототипов** облегчает доступ к часто используемым прототипам, храня набор предварительно созданных эталонных, готовых к копированию объектов. Простейшее хранилище может быть построено с помощью хеш-таблицы вида *имя-прототипа → прототип*. Но для удобства поиска прототипы можно маркировать и другими критериями, а не только условным именем.

Пример 1

Прототип позволяет производить точные копии объектов геометрических фигур, не привязываясь к их классам.

Все фигуры реализуют интерфейс клонирования и предоставляют метод для воспроизводства самой себя. Подклассы используют метод клонирования родителя, а затем копируют собственные поля в получившийся объект.




```
// Базовый прототип.Псевдокод
abstract class Shape is
    field X: int
    field Y: int
    field color: string

    // Конструктор прототипа.
    constructor Shape(source: Shape) is
        this()
        this.X = source.X
        this.Y = source.Y
        this.color = source.color

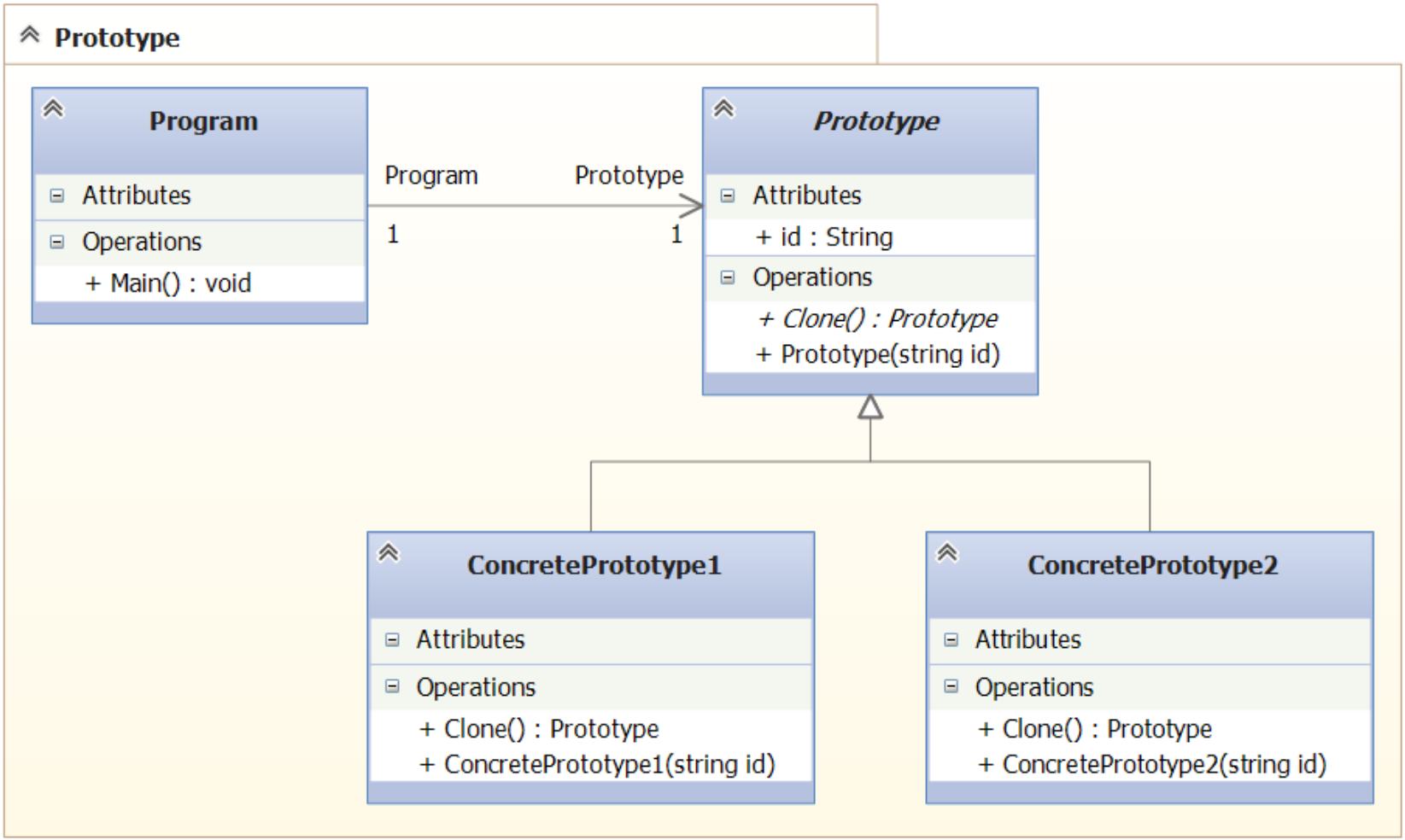
abstract method clone():Shape
```

```
class Rectangle extends Shape is
    field width: int
    field height: int

    constructor Rectangle(source: Rectangle) is
        // Вызов родительского конструктора нужен, чтобы
        // скопировать приватные поля,
        //объявленные в родительском классе.
        super(source)
        this.width = source.width
        this.height = source.height

    method clone():Shape is
        return new Rectangle(this)
```

Структура паттерна на UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Prototype prototype = null;
        Prototype clone = null;

        prototype = new ConcretePrototype1(1);
        clone = prototype.Clone();

        prototype = new ConcretePrototype2(2);
        clone = prototype.Clone();
    }
}
```

```
abstract class Prototype
{
    public int Id { get; private set; }

    public Prototype(int id)
    {
        this.Id = id;
    }

    public abstract Prototype Clone();
}
```

```
class ConcretePrototype1 : Prototype
{
    public ConcretePrototype1(int id)
        : base(id)
    {
    }

    public override Prototype Clone()
    {
        return new ConcretePrototype1(Id);
    }
}
```

```
class ConcretePrototype2 : Prototype
{
    public ConcretePrototype2(int id)
        : base(id)
    {
    }

    public override Prototype Clone()
    {
        return new ConcretePrototype2(Id);
    }
}
```

Участники

Prototype - **Прототип**:
Предоставляет интерфейс для клонирования себя.

ConcretePrototype - **Конкретный прототип**:
Реализует операцию клонирования себя.

Client - **Клиент**:
Клиент создает экземпляр прототипа. Вызывает на прототипе метод клонирования.

Отношения между участниками

Отношения между классами

Класс ConcretePrototype связан связью отношения наследования с абстрактным классом Prototype.

Отношения между объектами

Клиент вызывает на экземпляре-прототипе метод *Clone* и этот метод создает экземпляр-клон прототипа и возвращает ссылку на него.

Применимость

1. Код не должен зависеть от классов копируемых объектов.

Паттерн **Прототип** предоставляет клиенту общий интерфейс для работы со всеми прототипами. Клиенту не нужно зависеть от всех классов копируемых объектов, а только от интерфейса клонирования.

2. В наличии много подклассов, которые отличаются начальными значениями полей. Подклассы созданы, чтобы иметь возможность легко порождать объекты с определённой конфигурацией.

Паттерн прототип предлагает использовать набор прототипов, вместо создания подклассов для описания популярных конфигураций объектов.

Таким образом, вместо порождения объектов из подклассов, можно копировать существующие объекты-прототипы, в которых уже настроено внутреннее состояние. Это позволит избежать существенного роста количества классов в программе и уменьшить её сложность.

Шаги реализации

1. **Создать интерфейс прототипов** с единственным методом ***clone***. Если уже есть иерархия продуктов, метод клонирования можно объявить непосредственно в каждом из её классов.
2. **Добавить в классы будущих прототипов альтернативный конструктор, принимающий в качестве аргумента объект текущего класса**. Этот конструктор должен скопировать из поданного объекта значения всех полей, объявленных в рамках текущего класса, а затем передать выполнение родительскому конструктору, чтобы тот позаботился о полях, объявленных в суперклассе.
3. **Метод клонирования** обычно состоит всего из одной строки: вызова оператора **new** с конструктором прототипа. Все классы, поддерживающие клонирование, должны явно определить метод ***clone***, чтобы использовать собственный класс с оператором **new**. В обратном случае результатом клонирования станет объект родительского класса.
4. **Опция: создать центральное хранилище прототипов**. В нём удобно хранить вариации объектов, возможно, даже одного класса, но по-разному настроенных.

   Хранилище можно разместить в новом фабричном классе, либо в фабричном методе базового класса прототипов. Такой фабричный метод должен на основании входящих аргументов искать в хранилище прототипов подходящий экземпляр, а затем вызывать его метод клонирования и возвращать полученный объект.

   Прямые вызовы конструкторов объектов, заменить вызовами фабричного метода хранилища прототипов.

Преимущества и недостатки

+ Позволяет клонировать объекты, не привязываясь к их конкретным классам.
+ Меньше повторяющегося кода инициализации объектов.
+ Ускоряет создание объектов.
+ Альтернатива созданию подклассов для конструирования сложных объектов.
- Сложно клонировать составные объекты, имеющие ссылки на другие объекты.